

Investigating Task Scheduling on Intel's Three-Tier Arrow Lake-H

David Dawei Kan

University of Georgia

CSCI 6720 Computer Systems Architecture

Suchendra M. Bhandarkar

May 1, 2026

Abstract

Hybrid CPUs that mix two core types on a singular die have become the dominant CPU architecture, and the literature on operating-system scheduling has grown around the binary abstraction where: a thread is either a “performance” thread or an “efficiency” thread, and the scheduler picks one of two core types. Intel’s Arrow Lake-H, shipping in late 2024, breaks this abstraction by adding a physically separated third tier - two low-power efficiency cores (LP E) on a separate SoC tile, outside the compute-tile L3, on a different power domain. This paper reviews three influential approaches to hybrid-CPU scheduling (PIE [1], PMCSched [2], and HARP [3]) and asks whether the binary core-type assumption that all three share continues to be enough with the introduction of a third tier. We complement this review with a measurement study on a Core Ultra 9 285H running Fedora 43 (kernel 6.17): a three-point performance and energy surface across the performance (P), efficiency (E), and low-power efficiency (LP E) tiers on STREAM and four NAS Parallel Benchmark kernels, an Intel Thread Director observability check, and a comparison of four CPU placement policies on a mixed concurrent workload. Three findings stand out. First, the Linux Kernel structurally distinguishes the LP E island at three independent layers (scheduling-domain hierarchy, perf PMU name space, uncore PMU). On embarrassingly-parallel integer code (NPB EP) the E tier delivers 53% higher throughput and 43% less package energy than the P tier countering the conventional “P for performance, E for efficiency” ordering that the three reviewed schedulers encode. And while three-tier-aware placement beats single-tier oversubscription by 34%, three-way placement does not automatically improve on a two-tier P + E split for current mixed workloads. The LP E tier’s scheduling value is workload conditional, and the binary assumptions assumed by PIE [1], PMCSched [2] and HARP [3] need explicit cross-tile cost terms before they are able to be mapped onto Arrow Lake-H.

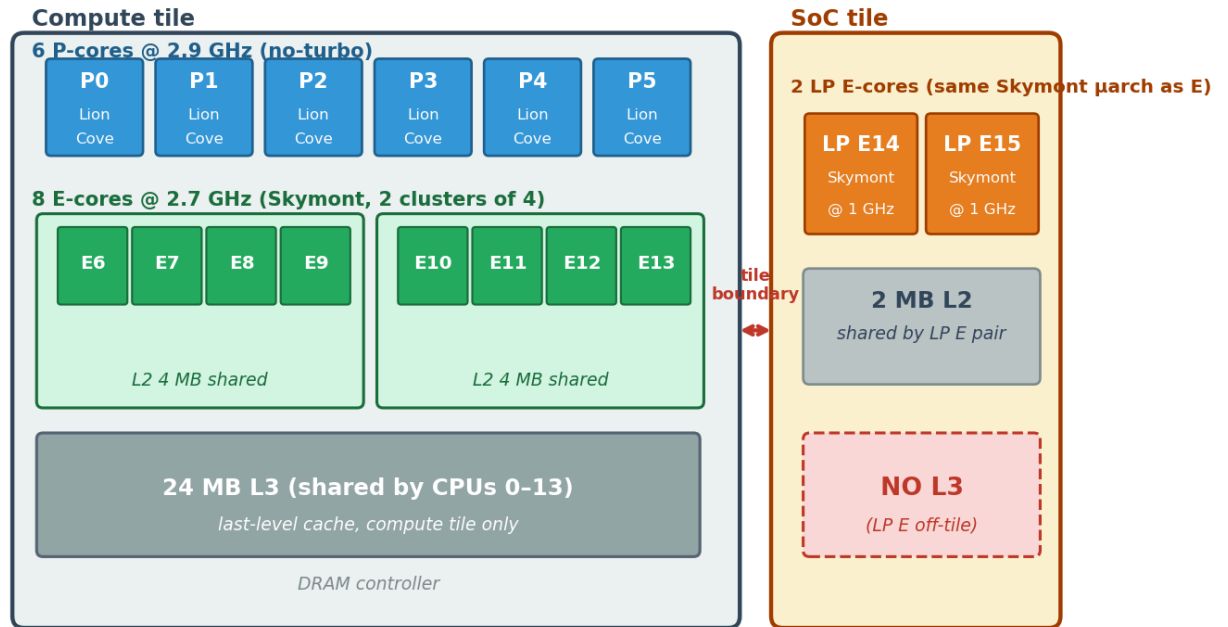
Table of Contents

Abstract	2
Table of Contents	3
Introduction	4
Literature Review	6
2.1 Scheduling Heterogeneous Multi-Cores through Performance Impact Estimation (PIE) - Van Craeynest et al., ISCA 2012	6
2.2 Flexible system software scheduling for asymmetric multicore systems with PMCSched: A case for Intel Alder Lake - Bilbao, Saez, Prieto-Matías, CCPE 2023	7
2.3 HARP: Energy-Aware and Adaptive Management of Heterogeneous Processors - Smejkal, Khasanov, Castrillon, Härtig, Middleware 2025	8
2.4 Limitations of the Two Tiered Assumption	10
Methodology	12
3.1 Hardware and Software	12
3.2 System Preparation	12
3.3 Benchmarks	12
3.4 Phase 1: Structural Characterization	13
3.6 Phase 2: Performance and Energy Surface	13
3.7 Phase 3: Thread Director Observability	14
3.8 Phase 4: Comparing Four CPU Placement Policies	14
Results	16
4.1 Reported Results	16
4.2 The Kernel Structurally Distinguishes the LP E Island	17
4.3 No Single Tier Dominates the Performance and Energy Surface	19
4.4 Intel Thread Director is Dormant on this Platform	23
4.5 Three-way Placement is not Automatic	24
Conclusions and Future Work	27
References	29

Introduction

The first commercial single-ISA heterogeneous CPU, ARM big.LITTLE, released in 2011 and gave operating system schedulers a new decision to make: each thread needs to be assigned not only a CPU but a core type as well. A decade later, every mainstream vendor ships hybrid silicon from Apple’s M-series since 2020, Intel’s Alder lake since late 2021, Meteor Lake in 2023, and ArrowLake in 2024. As such, the scheduling literature has grown around the same simplifying assumption that big.LITTLE introduced: there are two core types, “big”, and “little”, “P”, and “E,” “high-performance” and “energy efficient”, where the scheduler picks one of them per thread. Performance Impact Estimation (PIE) [1] formalizes this as a binary speedup prediction. PMCSched [2] reads Intel Thread Director’s class identifier as a per-thread P-vs-E label. HARP [3] indexes its resource vector and energy model by exactly two core types. None of these models were evaluated on hardware with more than two core types simultaneously.

Intel’s Arrow Lake-H breaks this binary assumption. The core Ultra 9 285H integrates 3 distinct core groups: 6 Lion Cove performance cores and eight Skymont efficiency cores on the compute tile sharing 24MB of last-level cache, and two Skymont efficiency cores on a separate SoC tile without a L3 connection at all. Figure 1 shows the schematic layout. The two on-tile and off-tile efficiency islands share the same microarchitecture but differ in physical locality, voltage, frequency envelope, and power domain. Linux 6.17 ships with structural awareness of this third tier building upon a decade of incremental hybrid-aware scheduler work in the kernel, summarized by Corbet [4] in his report on Brown and Neri’s 2022 Linux Plumbers Conference talk. This paper investigates whether this awareness changes the placement story.

Intel Core Ultra 9 285H — three core tiers across two physical tiles

Tile assignment from `/sys/devices/system/cpu/cpu*/cpufreq/cpuinfo_max_freq` and `lstopo`

Figure 1. Logical block diagram of the Intel Core Ultra 9 285H, showing the three core tiers and the compute/SoC tile boundary. Core counts, cache sizes, and the absence of an L3 connection on the SoC tile are derived from the system’s own `/sys/devices/system/cpu/*/cache/` topology and the kernel’s hybrid-PMU enumeration; microarch labels follow Intel’s Core Ultra 200H product documentation. This diagram is logical, and physical placement on the silicon is not asserted.

The two questions guide this work:

- Does the Linux kernel distinguish the LP E island as a third scheduling tier, or does it fold the two off-tile cores into the compute-tile E-core pool?
- Given what is exposed by the kernel, does a three-tier-aware placement policy meaningfully outperform the binary P vs E split that PIE, PMCSched, and HARP all assume?

Literature Review

2.1 Scheduling Heterogeneous Multi-Cores through Performance Impact Estimation (PIE)

- Van Craeynest et al., *ISCA 2012*

PIE [1] starts from the observation that the simplest heuristic for placing threads on a heterogeneous CPU: “memory-intensive code goes to small cores, compute-intensive code goes to big cores” is incorrect on both ends. The reasoning behind this heuristic is “based on the intuition that compute-intensive workloads benefit more from the high computational capabilities of a big core while memory-intensive workloads execute more energy-efficiently on a small core while waiting for memory” PIE [1]. The paper’s two motivating examples are *soplex*, which is memory-intensive but benefits from a big core because it has enough memory level parallelism (MLP) to fill a deep reorder window, and *calculix*, which is compute-intensive but runs nearly as well on a small core because its instruction-level parallelism (ILP) is low. The right placement axis, the authors argue, is not memory intensive, but rather the amount of MLP and ILP the workload can still exploit on the alternative core type.

To make this prediction, PIE samples the CPI stack of whichever core is currently executing a thread and predicts what the thread’s CPI would be on the other core type by combining the MLP ratio and ILP ratio terms. This results in a small extra hardware cost of 5 10-bit counters for a dependency-distance distribution and a 4-bit field per architectural register, totaling roughly 15 bytes of new state per core [1]. The scheduler invokes the predictor every 2.5 s and pays a 300-cycle migration cost when swapping a thread between cores [1].

PIE compares itself against two baselines. The first is the memory-dominance heuristic (memory-intensive threads to the small core and compute-intensive threads to the big core. The second is a sampling-based scheduler that periodically migrates threads between cores at runtime and keeps whichever placement runs faster. PIE’s reported gains are measured only on what the paper calls scheduling-sensitive workloads (SPEC CPU2006 mixes for which placement clearly favors one core type (workloads that either run almost as fast on the small core or run dramatically slower on it; middle-ground workloads are excluded because no placement choice helps them much). On a private-LLC config, PIE beats the heuristic by 5.5% in average throughput and the sampler by 8.7%; on a shared-LLC configuration, the gains decrease to 3.7% and 6.4% respectively. The paper is also honest about its error bars with 9% average and 35%

peak prediction error when estimating small-core behaviors from a big-core sample with 13% and 47% in the opposite direction [1] which isn't catastrophic but is also relatively large compared to the speedups they provide.

This paper, however, does have some limitations. First, PIE was never evaluated on physical hardware as the authors didn't have access to a big.LITTLE CPU. Second, the cross-core CPI prediction in PIE assumed that both core types shared the same cache hierarchy at every level and explicitly notes "that asymmetric caches require external techniques borrowed from a separate paper [Jaleel et al., ASPLOS 2011]" [1]. The authors acknowledge a 9% average / 35% maximum prediction error in the small-to-big direction and a 13% / 47% error in the reverse direction (§3.3, p.217). Section 4.2 notes that the formulation could in principle generalize to "any number of cores of any core type," but no multi-core-type evaluation appears in the paper.

2.2 Flexible system software scheduling for asymmetric multicore systems with PMCSched: A case for Intel Alder Lake - Bilbao, Saez, Prieto-Matías, CCPE 2023

PMCSched [2] addresses a practical problem in scheduler research where new hybrid-CPU features take years to cleanly land in mainline Linux, and patching the kernel just to evaluate a placement idea is extremely laborious. To address this problem, the authors built PMCSched as a loadable kernel module on top of their earlier PMCTrack tracing facility, hooking into the CFS through ftrace and static tracepoints [2]. This eliminates the need for a kernel patch as new policies are written as plugins and loaded at runtime.

The paper's main contribution is the Hybrid Scheduling Policy (HSP), which was implemented in three variants that differed in how they obtained a per-thread speedup factor (SF) (the predicted ratio of big-core to small-core throughput). HSP/TD reads the SF directly from Intel Thread Directors IA32_THREAD_FEEDBACK_CHAR register. HSP/B uses a PMC-based regression model executed only when the thread is run on the big core, caching results in a per-thread history table for reuse on small cores. HSP/BS (the core contribution) runs the regression on both core types using an additive-regression model over different processor stall cycle events [2].

PMCSched also documents one of the most-cited observability findings in the recent literature. On the I9-12900K Alder Lake Platform (8 Gold Cove P + 8 Gracemont E, Linux

5.16.9 on Debian 11), the `IA32_THREAD_FEEDBACK_CHAR` register “does report a valid class ID on P-cores most of the time, [but] always returns invalid class IDs for any thread running on E-cores” [2]. The authors dub this an “asymmetric observability” problem and frame their history-table workaround as compensation for this limitation. This history table caches a thread’s most recent valid P-core SF and is substituted whenever a thread is run on an E-core. The intel Thread Director itself classifies threads into four classes: Class 0 for general arithmetic and control flow, Class 1 and 2 for vector / AI / accelerator heavy code (with a 2.75x average speedup advantage on P-cores) , and Class 3 for spin-waiting code [2]. The per-class P/E speedup factors reported in PMCSched’s table 1 are 1.67, 1.97, 2.62, 1.31 for Classes 0, 1, 2, 3 respectively. This demonstrates that even the binary P/E gap cannot be generalized to a single number.

Three limitations do apply when trying to generalize these methods to the three cored cpus such as those in Arrow Lake-H. First, the evaluation only tests two core types (P and E) on a single die with a single shared L3 . Since Alder Lake doesn’t have a separate tile, the paper has no reason to model the cross-tile migration costs found in this newer hardware (HW). Second, both PMC-based variants assume the same HW counters are available on every core type. That holds on Alder Lake but isn’t guaranteed on later chips: on Arrow Lake-H specifically, the P-core PMU exposes events the E-core PMU does not (Section 4.2). Third, the history-table workaround only works if the P-cores return at least some valid class IDs to populate the cache. If every tier reads zero as Section 4.4 shows, as is the case on Arrow Lake-H, the workaround no longer has a fallback.

2.3 HARP: Energy-Aware and Adaptive Management of Heterogeneous Processors - Smejkal, Khasanov, Castrillon, Härtig, Middleware 2025

HARP [3] diverges from the traditional per-thread classification entirely. Its critique of PIE (and PMCSched) style schedulers is that they treat each thread in isolation and don’t communicate resource decisions back to the application; as such, they cannot exploit application level adaptivity. The authors propose Heterogeneity-aware Adaptive Resource Partitioning (HARP), a framework that establishes a two-way communication channel between applications and a userspace resource manager via the `libharp` library, exchanging protobuf message over Unix sockets [3].

The application-side abstraction is the operating point which is a tuple of an in-application configuration (e.g., OpenMP thread count, algorithm variant), a resource allocation expressed as an extended resource vector giving the number of cores and SMT lanes used per core type, and the associated utility and power values [3]. The example in the paper uses the vector [1, 2, 4] to mean one P-core lane plus two E-core lanes and four GPU lanes. This differs from just a binary big/little label. The resource manager then solves a Multiple choice Multi-dimensional Knapsack Problem (MMKP) across all the registered applications using a Lagrangian-relaxation approximation and selects the operating point per application that minimizes an energy-utility cost calculated by: $o[\zeta] = \left(\frac{o[p]}{o[v^*]} \right) * \left(\frac{1}{o[v^*]} \right)$, where $o[p]$ is attributed power and $o[v^*]$ is utility normalized to the maximum observed [3]. Decisions are enforced through `sched_setaffinity` on the kernel side and through callbacks into `libharp` on the application side. This allows OpenMP regions to retune their thread count or custom apps to switch algorithms.

HARP's evaluation runs on two platforms; an Intel Core i9-13900K Raptor Lake (8 P + 16 E) under custom-compiled Linux 6.4 with an Intel-Thread-Director patch-set, and an Odroid XU3-E ARM big.LITTLE board under Linux 6.6 with Energy-Aware Scheduling. The baselines differ per platform so HARP is compared against CFS + ITD on Intel and against EAS on the Odroid [3].

The benchmark set was chosen to span four workload classes [3]. From HPC, NPB v3.4.2 is run at class C on the Intel and class A on the Odroid. From parallel-programming runtimes, six Intel TBB applications (binpack, fractal, parallel-preorder, pi, primes, and seismic) were picked as a sample of the framework's building blocks. From machine learning, a HARP-instrumented TensorFlow Lite ran two image-recognition models, VGG and AlexNet, modified to scale their parallelism at runtime. And from the embedded domain (on the Odroid only) two Kahn-Process-Network applications- a Mandelbrot generator and an lms (Leighton-Micali-signature implementation) were run in both static topology and dynamic-task parallel forms to exercise HARP's custom application adaptivity hooks.

Harp documents a 12% reduction in execution time and a 30% reduction in energy on the Raptor Lake compared to CFS + ITD, and 12% / 25% on the Odroid compared to EAS [3]. To

clarify the origins of these performance gains, the authors ran two experiments where they disabled different parts of the system. In the first, they replaced HARP entirely with a simpler scheduler that just reads Intel Thread Director’s class IDs and uses them to decide where to place threads (the same idea inspired from PMCSched’s HSP/TD variant). The result of this was essentially indistinguishable from the baseline (1.02x speed and 1.04x energy) reproducing PMCSched’s [2] finding on Alder Lake. In the second experiment, the authors kept HARP’s full resource manager but turned off the calls back into the application so the application could no longer retune itself when its allocation changed. This made things noticeably worse than baseline where workloads took about 1.67x longer and used about 35% more energy on single application Intel [3] thus confirming that HARP’s gains came from the back and forth between the resource manager and the application and the manager on its own doesn’t help.

When generalizing to Arrow Lake-H, this matters in two ways. First, the math underneath HARP is built around two core types, as the power equation uses two coefficients (one per type) and the model HARP trains to estimate the operating points was fit in a 2d space [3]. Second, the application side feedback that led to the majority of the performance and efficiency gains has no notion of a third tier. Because of this, an OpenMP region tuned to pick a thread count for 8 P-cores and 16 E-cores has no way of making a sensible choice for the non binary $6P + 8E + 2LP$ E cores without the application code being rewritten.

2.4 Limitations of the Two Tiered Assumption

The three works span thirteen years and three different mechanisms (a runtime CPI prediction, a kernel-module classification using PMC regression, and userspace cooperative resource partitioning) but all share the same hardware assumptions of only two CPU core types (PIE on a single-die simulated big/little, PMCSched on Alder Lake P+E, HARP on Raptor Lake P+E and Odroid big.LITTLE). As a result, none of the three papers have a cost term that distinguishes intra-tile migrations (within a shared LLC) from cross-tile migration(between physically separated cache hierarchies) since none of the test platforms required such a distinction. And each depends on a thread-classification channel that is, on Arrow Lake-H, either weaker than expected (PIE’s ILP ratio collapses between same-microarchitecture tiers, since both compute-tile E and SoC-tile LP E are Skymont) or empirically dormant (PMCSched’s TD-based variant, as Section 4.4 shows). HARP’s libharp framework would in principle accept a third

coefficient, but the published power model and the regressor's training procedure are both indexed by core type in a two-dimensional configuration space, and adding a third dimension would require fundamental changes to the model.

Methodology

All code, scripts, raw measurement artifacts, and the figure-generation sources are available at: <https://github.com/StreetLamp05/arrow-lake-h-benchmarking.git>

3.1 Hardware and Software

Measurements were collected on a Single ASUS ROG Zephyrus G16 (Model Number: GU605CR-G16.U95070TI) with an Intel Core Ultra 9 285H: 6 Lion Cove P-cores, 8 Skymont E-cores on the compute tile, and 2 Skymont LP E-cores on the SoC tile with 32 GB of LPDDR5x-8400 memory. The benchmarks were run on Fedora 43 Workstation with kernel 6.17.1-300.fc43.x86_64 and gcc 15.2.1. The machine was on AC power with a fully charged battery each session.

3.2 System Preparation

To remove sources of variance that would otherwise drown out the cross-tier differences, the system was prepared in two stages prior to each measurement session. The per-boot script (scripts/pin_system.sh) sets the CPU frequency governor to performance on every CPU, disables Intel Turbo Boost via `/sys/devices/system/cpu/intel_pstate/no_turbo=1`, disables address-space layout randomization using `kernel.randomize_va_space=0`, and sets the I/O scheduler to none on every block device. With turbo disabled, each tier is held at its non-turbo ceiling (2.9 GHz on P, 2.7 GHz on E, 1.0 GHz on LP E) so any throughput differences between tiers reflect microarchitecture, core count, and physical locality rather than turbo speeds.

The per-session script (scripts/prep_session.sh) makes the RAPL energy_uj sysfs files world-readable (so the measurement loop can sample energy without prompting for sudo mid run), caches sudo credentials and ensures the msr kernel module is loaded for the Thread Director readback in Section 4.4 The page cache, dentries, and inodes are dropped (`echo 3 > /proc/sys/vm/drop_caches`) immediately before each individual benchmark run.

3.3 Benchmarks

Two benchmark suites are used, both compiled from source with `-O3 -fopenmp -mcmmodel=medium`:

- STREAM (McCalpin [5]) is built with `STREAM_ARRAY_SIZE=12000000`, sized at 288 MB per array — large enough that the working set spills out of the 24 MB L3 and the inner Triad loop measures DRAM bandwidth rather than cache bandwidth.

- NPB-OMP-C (the C+OpenMP port of the NAS Parallel Benchmarks [6]) is built at class B for four kernels: EP (compute-bound, embarrassingly parallel), CG (memory-latency-bound, sparse conjugate gradient), IS (bandwidth-bound, integer sort), and MG (mixed, multigrid).

3.4 Phase 1: Structural Characterization

The kernel-side topology evaluated in Section 4.2 is captured by two scripts. `scripts/topology_dump.sh` records `lscpu`, `lstopo` (from `hwloc`), the per-CPU cache hierarchy from `/sys/devices/system/cpu/cpu*/cache/index*`, the perf PMU list from `perf list pmu`, and `cpuid` per tier. `scripts/sched_domain_dump.sh` enables verbose scheduling-domain export (`/proc/sys/kernel/sched_domain/*`) and dumps `/proc/schedstat` together with `/sys/kernel/debug/sched/domains/`. The scheduling domain hierarchy in Table 1 is read from these dumps and the `cpumasks` are reproduced verbatim.

3.6 Phase 2: Performance and Energy Surface

The three-point surface in Section 4.3 is collected in four passes that share `measure_one.sh` but differ in workload and threadcount:

Pass	Workload	Thread count	Tiers	Runs per cell	Total runs
A	STREAM Triad	1 thread	P / E / LP E	20	60
B	STREAM Triad	6 / 8 / 2 (full tier)	P / E / LP E	20	60
C	NPB EP, CG, IS, MG (class B)	1 thread	P / E / LP E	3	36
D	NPB EP, CG, IS, MG (class B)	6 / 8 / 2 (full tier)	P / E / LP E	3	36

	B)				
--	----	--	--	--	--

In total, Phase 2 produces 192 individual runs. The aggregator (scripts/[aggregate.py](#)) goes through the artifact tree, parses each run’s energy.txt and perf.txt, and writes the canonical results/profiling/summary.csv as well as the figures used in Section 4.3.

The first run of each NPB kernel on the P tier is consistently $1.5\times$ – $35\times$ slower than the runs that follow, an effect not seen on the E or LP E tiers. We attribute it to Intel HWP P-state settling on CPU 0 after an idle period. Any run whose wall time is at least $1.5\times$ the median of its peers is flagged is_warmup=True and excluded from the means; six of the 192 runs in the published dataset were flagged this way, all of them P-tier first runs.

3.7 Phase 3: Thread Director Observability

The Thread Director check in Section 4.4 is a direct readback of three model-specific registers: IA32_HW_FEEDBACK_PTR (MSR 0x17D0), IA32_HW_FEEDBACK_CONFIG (MSR 0x17D1), and IA32_THREAD_FEEDBACK_CHAR (MSR 0x17D2) on the logical CPUs. This is done using scripts/td_check.sh using rdmsr from the msr-tools package.

3.8 Phase 4: Comparing Four CPU Placement Policies

The policy comparison in Section 4.5 fixes a workload and varies the placement rule. The workload is twelve concurrent processes: six single-threaded cg.B workers (memory-latency-bound), four single-threaded ep.B workers (compute-bound), and two stress-ng --io 1 workers (I/O-bound), the last of which are killed once all NPB workers complete. The four policies differ only in the CPU set each process class is pinned to at launch via taskset -c:

Policy	cg.B	ep.B	stress-ng
cfs	unpinned	unpinned	unpinned
all_P	CPUs 0–5	CPUs 0–5	CPUs 0–5
pe_split	CPUs 0–5	CPUs 6–13	CPUs 6–13
three_way	CPUs 0–5	CPUs 6–13	CPUs 14–15

Each policy is run five times. For each run, the page cache is dropped and any leftover NPB or stress-ng processes are swept before launch; package and core RAPL energy are bracketed across the entire run; the timer stops when the last NPB worker exits (the stress-ng workers are then killed and not counted toward wall time). All twelve workers are launched in the same shell rather than in a `$(...)` subshell, because `&` inside a subshell orphans the child process and breaks wait. The aggregator (`scripts/aggregate_4a.py`) applies the same $1.5\times$ outlier rule used in Phase 2; one `pe_split` run was flagged and is excluded from the mean. The remaining 19 runs feed Table 4 and Figure 8.

Results

This section evaluates the results reported by each of the three reviewed papers against the assumptions in Section 2.4, then presents the new measurement study on the Core Ultra 9 285H that tests those assumptions empirically. The benchmark choices follow prior-work conventions (NAS Parallel Benchmarks per [2]; STREAM per [5]; CFS as baseline per [3]) so that the results are comparable.

4.1 Reported Results

PIE [1] reports a 5.5% throughput improvement over memory-dominance scheduling and an 8.7% improvement over sampling-based scheduling, averaged across SPEC CPU2006 mixes for the type-I/III workload categories on the private LLC configuration. On the shared LLC configuration, the gains are smaller (3.7% and 6.4% respectively) [1]. The paper’s prediction error analysis is 9% average and 35% max error on small-to-big speedup prediction and 13% / 47% in the opposite direction. PMCSched [2] reports up to a 30% throughput improvement and up to a 6% reduction in averaged normalized turnaround time (ANTT) over the AARR baseline on twenty SPEC CPU mixes (Figure 8 of [2]). The most interesting internal comparison is that the PMC-based HSP/BS variant beats the Thread Director based HSP/TD by 22.9% average throughput and the paper states its hardware classification path is outperformed by its software classification class. HARP [3] reports a 12% execution time reduction and a 30% energy reduction on the Raptor Lake i9-13900K against CFS + ITD, and a 12% / 25% reduction on the Odroid against EAS, averaging to a 12% / 28% advantage across both platforms [3]. Figure 2 reproduces these deltas for reference.

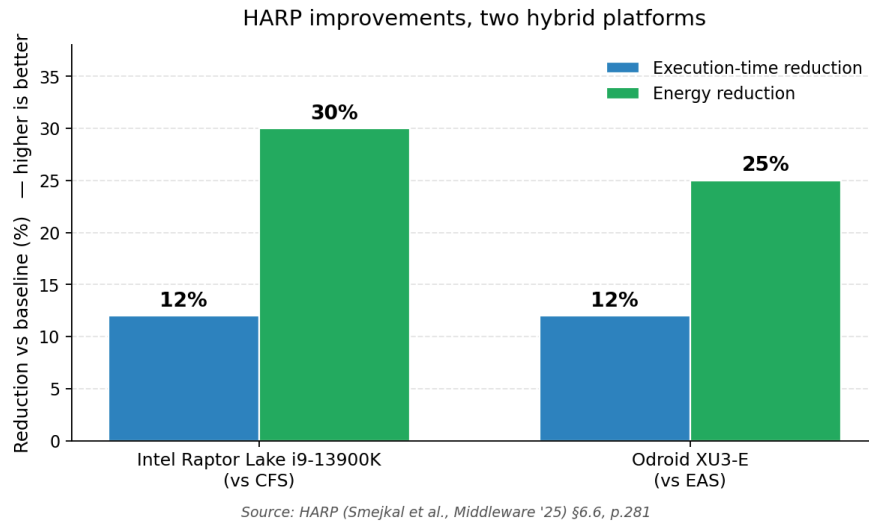


Figure 2. HARP’s reported execution time and energy reductions on its two evaluation platforms averaged across the benchmark suite [3]

As each was measured on different hardware, against different baselines, and on different workload sets, the three sets of numbers can’t be compared directly. However, what they share is that each was obtained on HW with at most two CPU core types, so generalization to a three-tier hybrid is an open question.

4.2 The Kernel Structurally Distinguishes the LP E Island

Figure 3 shows the logical topology of the Core Ultra 9 285H as reported by stopo (hwloc) - the machine-generated counterpart to the schematic in Figure 1. The two-tile layout is visible: P-cores 0-5 (Lion Cove, with private 3MB L2 each) and E-cores (Skymont, in two 4 core clusters each sharing a 4 MB L2) all share the 24 MB L3 on the compute tile, while the LP E-cores 14-15 sit on the SoC tile with their own 2MB L2 and no L3 connection. The subsequent topology arguments in this section are direct readings of this physical layout through the kernel’s /sys and /proc interfaces.

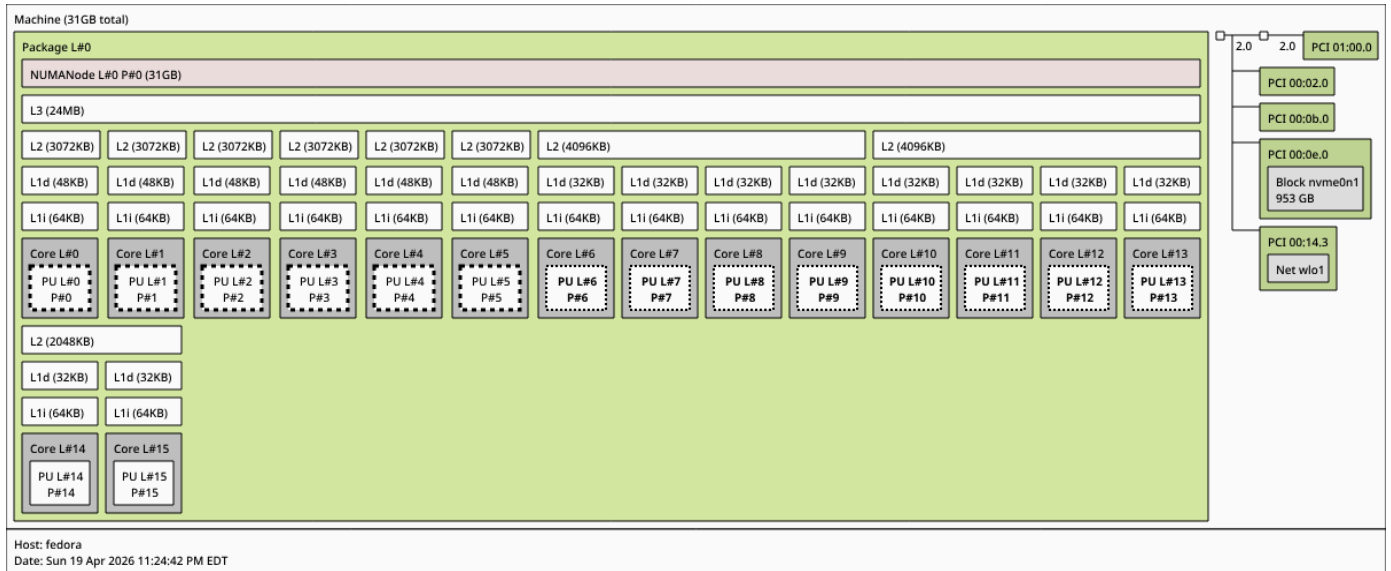


Figure 3. Topology of the Intel Core Ultra 9 285H, generated by lstopo from hwloc. Note the disjoint L3 (compute tile only) and the LP E-cores’ separate L2.

The Linux kernel exposes its scheduling-domain hierarchy through `/proc/schedstat` and through `/sys/kernel/debug/sched/domains/`. Parsing through the per-domain CPU masks gives the structures shown in Table 1.

Table 1. Scheduling-domain hierarchy per tier on the Core Ultra 9 285H

Tier	Domain levels (cpumasks in hex)
P	MC (0x3fff = CPUs 0–13) → PKG (0xffff = 0–15)
E 0–3	CLS (0x03c0 = CPUs 6–9) → MC (0x3fff) → PKG (0xffff)
E 4–7	CLS (0x3c00 = CPUs 10–13) → MC (0x3fff) → PKG (0xffff)
LP E	CLS (0xc000 = CPUs 14–15) → PKG (0xffff)

From this we see that first, the two LP E cores have their own cluster (CLS) domain separate from the two E-cluster CLS domains on the compute tile. Second, the LP E cores are excluded from the multicore (MC, last-level-cache) domain. The MC mask 0x3fff covers CPUs 0-13 but not 14-15. The kernel therefore never asks an LP E core to participate in load balancing

against a peer that shares its LLC as no such peer exists. Cache topology, exposed through `/sys/devices/system/cpu/cpu*/cache/index*`, confirms the underlying reason: the 24 MB L3 boundary is shared across CPUs 0-13 only; the LP E cores on the SoC tiles don't have an L3 connection.

The kernel's structural distinction is shown at two other layers. First the perf subsystem exposes three separate CPU PMU namespaces: `cpu_core` for the P tier (Lion Cove), `cpu_atom` for the compute-tile E tier (Skymont), and `cpu_lowpower` for the LP E tier. Additionally, two separate uncore PMUs (`uncore_cncu` for the compute tile, `uncore_sncu` for the SoC tile). Comparison of the event lists exposed by `cpu_atom` and `cpu_lowpower` shows that the two PMUs advertise identical event surfaces, evidence that the LP E cores share the same Skymont microarchitecture with the compute-tile E cores. The two Skymont PMUs also expose fewer events than the Lion Cove `cpu_core` PMU. The P tier's PMU has a set of top-down events (`slots`, `btopdown-br-mispredict`, `topdown-fetch-lat`, `topdown-heavy-ops`, and `topdown-mem-bound`) that neither E nor LP E exposes. This matters for the reviewed schedulers: PIE's [1] ILP-distance counters and PMCSched's [2] PMC-stall regression both need counters that, on Arrow Lake-H, exist only on the P tier.

4.3 No Single Tier Dominates the Performance and Energy Surface

Single-threaded STREAM Triad delivers 24.6 GB/s on P, 24.2 GB/s on E and 11.5 GB/s on LP E (means over 19-20 runs per tier; coefficients of variation below 1.5%). The P and E tiers fall within 1.7% of each other on per-core bandwidth despite the different microarchitectures. This is because on the compute tile the DRAM path is the bottleneck and a single Lion Cove core extracts relatively the same amount of bandwidth from it as a single Skymont core. The LP E tier (which is outside the compute tile's L3) operates at 47% of compute-tile per-core bandwidth, serving as evidence of the L3/ tile boundary identified in Section 4.2.

At full-tier thread counts (Figure 4), P reaches 69.1 GB/s on six threads ($2.81\times$ scale-up), E plateaus at 45.5 GB/s on eight threads ($1.88\times$), and LP E reaches 13.4 GB/s on two threads ($1.18\times$). Expressed as bandwidth per package watt, the ranking counters the conventional efficiency intuition: P delivers 3.78 GB per package-joule, E delivers 3.09, and LP E delivers 1.63. For bandwidth-bound throughput work, the "low-power" tier is the least energy-efficient choice, because its longer wall time keeps the rest of the package drawing idle current.

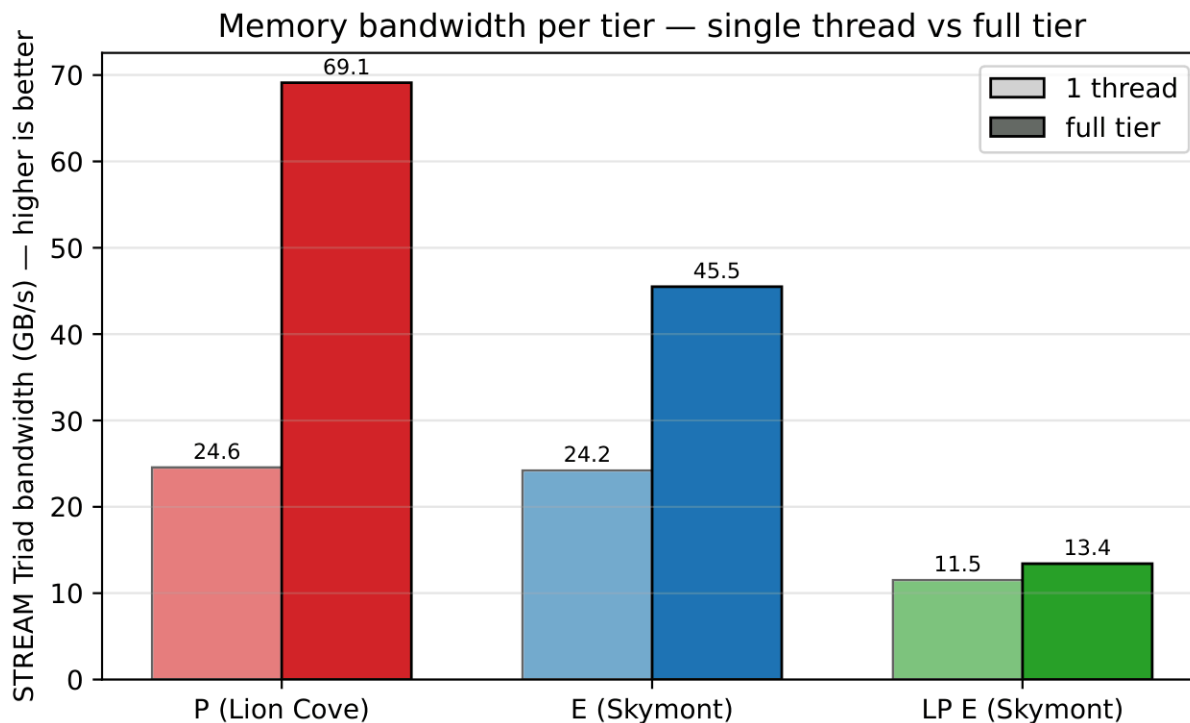


Figure 4. STREAM Triad memory bandwidth per tier, single-thread vs full-tier.

Figure 5 shows full-tier speedup of each tier relative to the P tiers six-thread baseline across the four NPB kernels, and Table 2 lists the underlying wall-time and energy ratios.

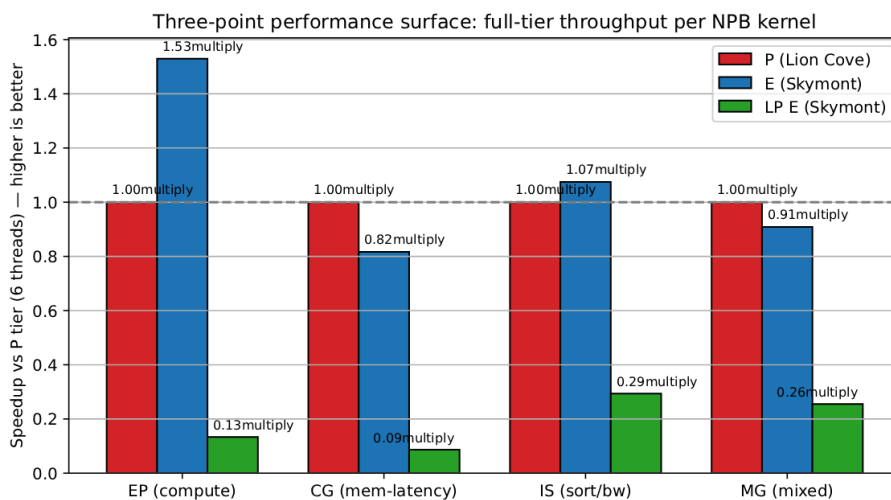


Figure 5. Three-point performance surface: full-tier speedup of each tier relative to the P tier's six-thread baseline across NPD EP, CG, IS, and MG (class B)

Kernel	Tier (threads)	Wall (s)	Wall speedup vs P	Pkg energy ratio vs P
EP	P (6t)	13.5	1.00	1.00
EP	E (8t)	8.85	1.53 *	0.57 *
EP	LP E (2t)	101	0.13	3.70
CG	P (6t)	6.96	1.00	1.00
CG	E (8t)	8.53	0.82	1.11
CG	LP E (2t)	80.0	0.087	3.95
IS	P (6t)	5.41	1.00	1.00
IS	E (8t)	5.03	1.07	0.94
IS	LP E (2t)	18.4	0.29	3.06
MG	P (6t)	2.15	1.00	1.00
MG	E (8t)	2.37	0.91	1.08
MG	LP E (2t)	8.41	0.26	2.61

Table 2. Full-tier NPB results, all three tiers. Wall-time speedup is reported relative to the P tier’s six thread baseline (> 1 means the tier is faster than P). Package energy is reported as a ratio to the P tier’s per-run energy

The starred (*) row is the result that most directly contradicts the binary core-type assumption shared by PIE [1], PMCSched [2], and HARP [3]. On EP (embarrassingly-parallel integer code with no memory work) the eight-threaded E tier is both 53% faster and 43% lower energy than the six-threaded P tier. Eight skymont cores at 2.7 GHz outproduce six Lion Cove cores at 2.9 GHz when the workload has no dependency chains long enough to exploit Lion

Cove’s wider out-of-order window. Any scheduler with a fixed "P is faster than E" prior (PIE’s binary speedup prediction, PMCSched’s class-ID-driven SF, HARP’s two-coefficient power model) will place EP on the P tier by default and concede both axes of the trade-off to a scheduler that does the opposite.

The three other NPB kernels show why the assumption is so robust. P wins by 22% on CG, by 10% on MG, and ties E on IS. The “P > E for performance” intuition is correct for $\frac{3}{4}$ kernels. However, the fourth (easiest to parallelize, most relevant to throughput focused servers) is incorrect by a significant amount.

The LP E tier completes the surface but does not win it. Across the four NPB kernels, LP E is between $2.6\times$ and $4.8\times$ slower than the faster compute-tile tier and consumes between $2.6\times$ (MG) and $4.0\times$ (CG) more package energy per run. Figure 6 shows: on every NPB kernel, the LP E bar is the tallest. None of the four kernels makes a case for LP E inclusion in an active placement set for performance or energy.

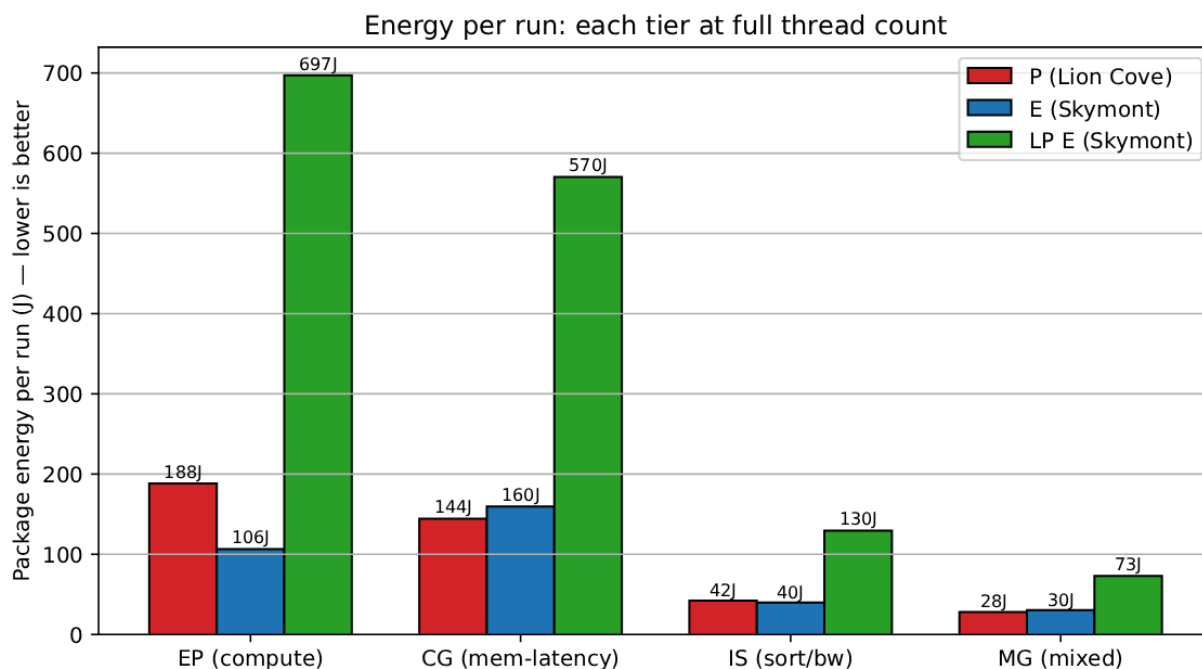


Figure 6. Per-tier package energy per full-tier run, NPB kernels (class B). LP E is the highest-energy choice on every kernel despite its lower instantaneous power.

4.4 Intel Thread Director is Dormant on this Platform

PMCSched [2]’s most-cited observability finding on Alder Lake is that the `IA32_THREAD_FEEDBACK_CHAR` register (MSR 0X17D2) is read-valid on P-cores most of the time and read-invalid on E-cores essentially all the time. We replicated the experiment on Arrow Lake-H and found that reading `IA32_THREAD_FEEDBACK_CHAR` on every logical CPU returns `0x0000000000000000` on every tier under every workload tested. The full per-CPU readback is in `results/td/msr_check.txt` and is reproduced in summary in Table 3.

CPUs	Tier	Readback (every CPU)
0–5	P	0x0000000000000000, valid bit clear
6–13	E	0x0000000000000000, valid bit clear
14–15	LP E	0x0000000000000000, valid bit clear

Table 3. Per-CPU `IA32_THREAD_FEEDBACK_CHAR` readback.

CPUID leaf 0x06 EAX returns `0x00df8ff7`, which has bit 19 (Hardware Feedback Interface) and bit 23 (Intel Thread Director) both set, and the kernel identifies “ArrowLake-H Hybrid events” on boot. However, the activation is missing: MSR `0x17d0` (`IA32_HW_FEEDBACK_PTR`) and `0x17d1` (`IA32_HW_FEEDBACK_CONFIG`) also returns zero on every CPU, meaning the kernel has not allocated a feedback-table page or written its address into the pointer register.

The impact on the three reviewed schedulers is as follows. PMCSched [2]’s HSP/TD variant cannot read what it consumes and the history-table workaround for invalid E-core readings has nothing to cache since the P-cores also return zero. HARP [3]’s ITD baseline runs against a custom Linux 6.4 patch-set that hasn’t propagated to Fedora’s stock 6.17 kernel for this CPU family. Finally, PIE [1]’s PMC needs are smaller and are met on the P tier, but its ILP-ratio formulation cannot distinguish E from LP-E as both microarchitectures are Skymont.

4.5 Three-way Placement is not Automatic

Section 4.3 establishes that the three-tier performance surface is not a partial order, and Section 4.4 establishes that the hardware classification channel that the reviewed schedulers consume is dormant on this platform. The remaining empirical question is whether a placement policy that uses all three tiers explicitly outperforms a two-tier P + E split on a representative mixed workload.

We compared four CPU placement policies on the mixed workload defined in Section 3.8: cfs (stock placement), all_P (all 12 workers on the P tier), pe_split (cg -> P, ep -> E, I/O -> E), and three_way (cg -> P, ep -> E, I/O -> LP E). Each policy ran five times; one pe_split outlier was excluded by the rule from Section 4.3. Results are shown in Table 4 and Figure 7.

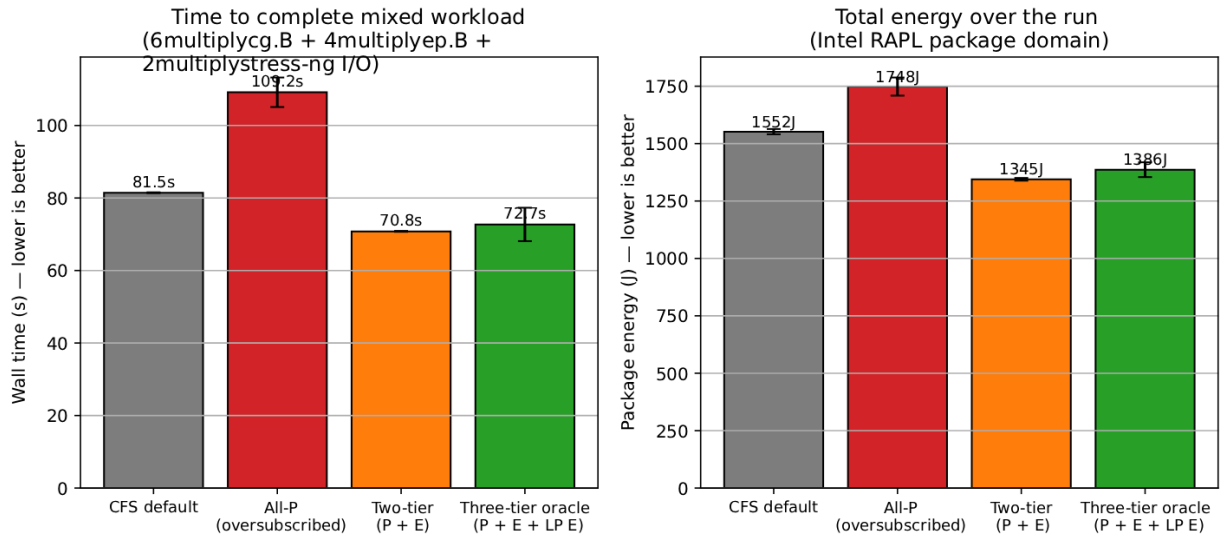


Figure 7. Four-policy comparison on the mixed workload. Left bar group: mean wall time per policy. Right bar group: mean package energy per policy. Error bars show ± 1 standard deviation; outliers excluded as described in the text.

Policy	Wall (s)	Pkg energy (J)	Δ wall vs CFS	Δ energy vs CFS
all_P	109.16 $\pm$ 4.07	1748 $\pm$ 39	+34.0%	+12.7%
cfs	81.46 $\pm$ 0.11	1552 $\pm$ 12	baseline	baseline

pe_split	70.80 $\pm$ 0.12	1344 $\pm$ 6	-13.1%	-13.4%
three_way	72.71 $\pm$ 4.61	1386 $\pm$ 32	-10.8%	-10.7%

Table 4. Four-policy comparison on the mixed workload (mean $\pm$ standard deviation across runs).

Three results require interpretation. First, the all_P policy is the worst: placing all twelve workers onto the six P-cores costs 34% wall time and 13% energy versus stock CFS. The kernel's three-tier distinction is therefore not cosmetic as recognizing the tiers and distributing work across them is worth a measurable 34% wall-time improvement over ignoring them.

Second, both pe_split and three_way clearly beat CFS by approximately 13% on wall time and energy. The mechanism is the same in both cases: matching each NPB workload class to its Section-4.3-preferred tier (cg $\rightarrow$ P, ep $\rightarrow$ E) captures most of the available improvement. Stock CFS does balance work across tiers by load but does so without consulting workload-class preferences, and in our mix it places enough cg workers on E and ep workers on P to cost the 13% gap.

Finally, three_way does NOT improve on pe_split. The two tier policy beats the three-tier by 1.91s (2.6% on wall time, by 41.9 J (3.0%) on energy, and by a factor of nearly 40x on run-to-run variance (wall stdev 0.12 s vs 4.61 s). The reasoning behind this is due to the type of workload, the mixed workload's critical path is defined by the ten CPU-bound NPB workers, the E tier already has slack with only four embarrassingly parallel workers running on eight cores, and the two stress-ng I/O workers are CPU-cheap enough that whether they live on E or LP E does not change the wall-time.

The implication for PIE [1], PMCSched [2], and HARP [3] is not that the LP E tier should be ignored. Conversely, this implies that the three-way placement is workload conditional in a way that the binary-tier literature has not had to face. Three-way would only clearly win when the workload has a natural LP E consumer (such as a long-running task whose completion contributes to the wall-time critical path and whose throughput requirements are low enough that the LP E's throughput suffices. The pure-compute mix used in our experiment did not have a

consumer that fit those requirements, and routing I/O onto LP E did not produce measurable benefits while adding a variance penalty.

Conclusions and Future Work

The paper asked whether the binary core-type abstraction shared by PIE [1], PMCSched [2], and HARP [3] continues to be adequate for an Intel hybrid platform that has three physically distinct core groups. The answer has three parts.

First, structurally, Linux 6.17 already distinguishes the LP E island as a third tier (drawing the boundary at the scheduling domain hierarchy, the perf PMU namespace, and the uncore PMU namespace) and it coincides with the L3/ tile boundary in the silicon. The three reviewed schedulers therefore do not have to confront a kernel that hides the third tier as the OS already exposes it and any extension can be built upon what `/sys` and `/proc` already report.

Second, on the performance and energy surface, the binary “P for performance, E for efficiency” ordering that all three reviewed schedulers use is incorrect at least one important workload class: full-tier NPB EP runs 53% faster and uses 43% less package energy on the E tier than on the P tier on Arrow Lake-H. PIE’s binary speedup prediction, PMCSched’s class-ID-driven SF, and HARP’s two-coefficient power model would misplace EP on the P tier and concede the simultaneous throughput-and energy improvements. The LP E tier, meanwhile, is consistently the least throughput-competitive and the least energy efficient option across the workload classes typically used to evaluate hybrid-CPU schedulers. The standard NPB workload set doesn’t by itself justify routing work to the LP E tier.

Finally, on scheduling policy, recognizing the tier structure at all is worth approximately 34% walltime over a single-tier oversubscription on a mixed workload, and matching each workload class to its preferred tier is worth an additional 13% over stock CFS. But moving from a two-tier P+E split to a three-tier P + E + LP E split doesn’t automatically provide improvements for current mixed workloads. LP E’s contribution in our experiment was net-negative once run-to-run variance was taken into account. The value in LP E tier’s scheduling would therefore be conditional on the workload having a natural LP E critical-path consumer, which the current workload classes in literature (HPC kernels, SPEC mixes, server-style request processing) don’t seem to provide.

Some future directions from these findings are as follows. First, workloads whose three-way placement does beat two-way to find the workload where the inclusion of the third tier

is justified should be identified/ created. Second, extend the intel_hfi_thermal driver's CPU-match list to include CPU family 6 model 197, so Thread Director activates on stock kernels for this CPU model. And third, extend the cost models in PIE, PMCSched, and HARP from binary to ternary, with an explicit cross-tile migration cost term that captures the LLC-cold startup penalty between the compute tile and the SoC tile.

References

- [1] K. Van Craeynest, A. Jaleel, L. Eeckhout, P. Narvaez, and J. Emer. "Scheduling heterogeneous multi-cores through performance impact estimation (PIE)." In 2012 39th Annual International Symposium on Computer Architecture (ISCA), pp. 213–224, 2012. doi:10.1109/ISCA.2012.6237019.
- [2] C. Bilbao, J. C. Saez, and M. Prieto-Matías. "Flexible system software scheduling for asymmetric multicore systems with PMCSched: A case for Intel Alder Lake." *Concurrency and Computation: Practice and Experience*, vol. 35, no. 25, e7814, 2023. doi:10.1002/cpe.7814.
- [3] T. Smejkal, R. Khasanov, J. Castrillon, and H. Härtig. "HARP: Energy-Aware and Adaptive Management of Heterogeneous Processors." In *Proceedings of the 26th International Middleware Conference (Middleware '25)*, Nashville, TN, USA, Dec. 2025, pp. 270–284. doi:10.1145/3721462.3770774.
- [4] J. Corbet. "Hybrid scheduling gets more complicated." *LWN.net*, Sep. 30, 2022. <https://lwn.net/Articles/909611/>. Conference report on Linux Plumbers Conference 2022 talks by L. Brown and R. Neri.
- [5] J. D. McCalpin. "Memory bandwidth and machine balance in current high-performance computers." *IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter*, Dec. 1995. <http://www.cs.virginia.edu/stream/>.
- [6] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrisnan, and S. K. Weeratunga. "The NAS Parallel Benchmarks." *International Journal of Supercomputer Applications*, vol. 5, no. 3, pp. 63–73, 1991. doi:10.1177/109434209100500306.